KAFKA — COMPLETE HANDS-ON LAB

Simple step-by-step training | Docker Compose + CLI + Kafka UI + Python scripts

Setup used throughout: docker-compose.yml | Kafka container: kafka | Kafka UI: http://localhost:8085 | Kafka CLI: localhost:19092 | Python on host: localhost:9092

Use a separate terminal for each long-running producer or consumer. Stop one with Ctrl+C. Commands below run from your computer unless marked “inside Kafka container.”

Lab 0 — Start Kafka and Kafka UI

1. Open a terminal in the folder containing docker-compose.yml.
2. Start only Kafka and Kafka UI:

```
docker compose up -d kafka kafka-ui
docker compose ps
```

1. Open http://localhost:8085 and select local-kafka.
2. If Kafka is not running, check logs:

```
docker compose logs --tail=80 kafka
```

Concept: Kafka stores event records; Kafka UI displays topics, partitions, messages and consumer groups.

Lab 1 — One producer → one consumer (CLI)

Goal: Send messages from one producer and receive them in one consumer.

1. Create a topic with one partition:

```
docker exec kafka /opt/kafka/bin/kafka-topics.sh --create --if-not-exists --topic lab-one --partitions 1 --replication-factor 1 --bootstrap-server localhost:19092
```

Terminal 1 — start the consumer:

```
docker exec -it kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:19092 --topic lab-one --from-beginning
```

Terminal 2 — start the producer:

```
docker exec -it kafka /opt/kafka/bin/kafka-console-producer.sh --bootstrap-server localhost:19092 --topic lab-one
```

Type each line and press Enter:

```
order1,100
order2,200
order3,300
```

Expected: Terminal 1 prints the three messages. Kafka keeps the messages even after the consumer reads them.

UI check: Topics → lab-one → Messages. Look at message values and offsets 0, 1, 2 (if new topic).

Lab 2 — One producer → two consumers (different groups)

Goal: Show fan-out: both consumers receive the same events when they use different consumer groups.

1. Use topic lab-one. Open two new terminals.

2. Start Consumer A with group sales:

```
docker exec -it kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:19092 --topic lab-one --group sales --consumer-property auto.offset.reset=earliest
```

Start Consumer B with group analytics:

```
docker exec -it kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:19092 --topic lab-one --group analytics --consumer-property auto.offset.reset=earliest
```

Use the producer from Lab 1 to send order4,400 and order5,500.

Expected: Both groups receive new records independently. Different consumer groups each get their own copy of the stream.

UI check: Consumer Groups → sales and analytics; inspect each group separately.

Lab 3 — One producer → two consumers (same group, 1 partition)

Goal: Learn why adding consumers does not always increase parallelism.

1. Open two terminals; start the following command in BOTH:

```
docker exec -it kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:19092 --topic lab-one --group shared-one --consumer-property auto.offset.reset=earliest
```

Send order6,600 from the Lab 1 producer.

Expected: Only one of the two consumers reads the new message. One partition can be assigned to only one consumer within a group; the other consumer stays idle.

UI check: Consumer Groups → shared-one → Members / Partitions; look for one active assignment.

Lab 4 — Topic with multiple partitions

Goal: See partitions and their independent offsets.

```
docker exec kafka /opt/kafka/bin/kafka-topics.sh --create --if-not-exists --topic orders-multi --partitions 3 --replication-factor 1 --bootstrap-server localhost:19092
docker exec kafka /opt/kafka/bin/kafka-topics.sh --describe --topic orders-multi --bootstrap-server localhost:19092
```

Expected: Partition 0, 1 and 2; each has its own offset sequence. Replication factor 1 means this single-broker lab has no broker-failure tolerance.

UI check: Topics → orders-multi → Overview / Partitions. Confirm 3 partitions.

Lab 5 — Produce messages with keys (partition routing)

Goal: Events with the same key normally go to the same partition, while different keys may or may not.

Open producer in a terminal (one line to avoid shell formatting mistakes):

```
docker exec -it kafka /opt/kafka/bin/kafka-console-producer.sh --bootstrap-server localhost:19092 --topic orders-multi --property parse.key=true --property key.separator=:
```

Enter:

```
customer1:order101,500
customer2:order102,700
customer1:order103,900
```

```
customer3:order104,300
```

Read keys and partitions in a different terminal:

```
docker exec -it kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:19092 --topic orders-multi --from-beginning --property print.key=true --property print.partition=true
```

Expected: customer1 records go to the same partition. Different keys are NOT guaranteed to choose different partitions. If the producer reports a separator error, copy the command exactly, including --property; use the Python producer in Lab 11 for guaranteed partition placement.

UI check: Topics → orders-multi → Messages; inspect key and partition columns.

Lab 6 — Two consumers in the SAME group (3 partitions)

Goal: Demonstrate load sharing with multiple partitions.

Terminal 1:

```
docker exec -it kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:19092 --topic orders-multi --group team-a --consumer-property auto.offset.reset=earliest
```

Terminal 2: run the SAME command. Keep both running. Produce more messages from Lab 5.

Expected: Three partitions are shared across two consumers (typically 2+1); messages are not duplicated within the group.

UI check: Consumer Groups → team-a → Members / Partitions; observe which consumer owns each partition.

Lab 7 — Rebalancing

1. Keep the two team-a consumers running.
2. Stop Consumer 2 with Ctrl+C.
3. Refresh Kafka UI and wait for the group to stabilize.
4. Send another message from the producer.

Expected: Consumer 1 takes ownership of all three partitions and continues reading. Restart Consumer 2 to trigger another rebalance.

UI check: Consumer Groups → team-a → Members; watch member count and partition assignments change.

Lab 8 — Multiple consumer groups

Goal: Two teams consume the same topic independently.

Leave team-a running and start a consumer in a new terminal:

```
docker exec -it kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:19092 --topic orders-multi --group team-b --consumer-property auto.offset.reset=earliest
```

Send another record using the producer. Expected: team-a gets one copy within its group, and team-b also gets a copy. Each group tracks its own offsets.

UI check: Consumer Groups → team-a and team-b; compare membership and lag.

Lab 9 — Offsets and consumer lag

1. Stop all team-a consumers with Ctrl+C.
2. Send five new messages to orders-multi using the producer.
3. Refresh Consumer Groups → team-a.
4. Observe lag (when committed offsets exist).
5. Restart one team-a consumer and watch lag reduce after commits.

Concept: Lag = log-end offset minus committed group offset, per partition. Offset numbers are per partition, not global. Consumer commits can take a few seconds.

```
docker exec kafka /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server localhost:19092 --describe --group team-a
```

Lab 10 — Produce messages directly in Kafka UI

1. Open Topics → orders-multi → Messages.
2. Click Produce Message.
3. Enter Key = customer5 and Value = order105,150.
4. Send and refresh Messages.
5. Repeat with customer5 and customer6.
6. Compare their partitions. If your UI offers a partition selector, choose a specific partition for an explicit routing demo.

Expected: Records appear in Kafka UI; equal keys normally route together. UI labels vary slightly by release.

Lab 11 — Python producer: guaranteed multiple partitions

Goal: Use a script to send JSON orders explicitly to partitions 0, 1 and 2.

On your COMPUTER (not inside Kafka), install the Python client:

```
python -m pip install kafka-python
```

Create producer.py with this exact content:

```
from kafka import KafkaProducer
import json
import time

producer = KafkaProducer(
    bootstrap_servers="localhost:9092",
    value_serializer=lambda v: json.dumps(v).encode("utf-8")
)

for i in range(1, 11):
    order = {"order_id": i, "customer_id": f"C{i}", "amount": i * 100}
    partition = (i - 1) % 3
    metadata = producer.send("orders-python", value=order, partition=partition).get(timeout=10)
    print(f"Order {i} -> Partition {metadata.partition}, Offset {metadata.offset}")
    time.sleep(1)

producer.flush()
producer.close()
```

Create the topic, then run the script:

```
docker exec kafka /opt/kafka/bin/kafka-topics.sh --create --if-not-exists --topic orders-python --partitions 3 --replication-factor 1 --
bootstrap-server localhost:19092
python producer.py
```

Expected: Orders 1,4,7,10 → P0; 2,5,8 → P1; 3,6,9 → P2. Offsets depend on previous messages.

UI check: Topics → orders-python → Messages; filter P0, P1, P2 and verify JSON records.

Lab 12 — Python consumer

Create consumer.py:

```
from kafka import KafkaConsumer
import json

consumer = KafkaConsumer(
    "orders-python",
    bootstrap_servers="localhost:9092",
    group_id="python-group",
    auto_offset_reset="earliest",
    value_deserializer=lambda v: json.loads(v.decode("utf-8"))
)

print("Waiting for orders...")
for msg in consumer:
    print(f"Partition={msg.partition} Offset={msg.offset} Order={msg.value}")
```

Run:

```
python consumer.py
```

Expected: Prints JSON orders with partition and offset. Run producer.py again to see new events. Stop with Ctrl+C.

UI check: Consumer Groups → python-group; inspect group members and lag.

Lab 13 — Two Python consumers, same group

1. Open two terminals on your computer.
2. Run python consumer.py in BOTH terminals.
3. Run python producer.py in a third terminal.
4. Observe which orders each consumer prints.
5. Stop one consumer and run the producer again.

Expected: Each partition belongs to one Python consumer in python-group. After one exits, the remaining consumer takes over all partitions.

UI check: Consumer Groups → python-group → Members / Partitions; compare before and after stopping one consumer.

Lab 14 — Retention (optional, separate topic)

1. In Kafka UI create retention-demo with 1 partition and replication factor 1.
2. Produce two messages.

3. Open its Config tab and set retention.ms = 60000 (if editable).

4. Wait and refresh Messages.

Concept: 60 seconds makes old data eligible for deletion; removal is asynchronous and may take longer. Do not change retention on your main lab topics.

Lab 15 — Kafka → PySpark streaming (optional extension)

Goal: Spark reads Kafka messages continuously. This requires a Spark Kafka connector matching the installed Spark/Scala version; the Jupyter image is tagged latest, so verify its versions first.

```
docker compose up -d jupyter
docker exec jupyter spark-submit --version
```

Open http://localhost:8888 (token: training123). In a notebook, after installing the compatible spark-sql-kafka-0-10 connector for your Spark version, use:

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("KafkaLab").getOrCreate()
df = (spark.readStream.format("kafka")
    .option("kafka.bootstrap.servers", "kafka:19092")
    .option("subscribe", "orders-python")
    .option("startingOffsets", "earliest").load())
query = (df.selectExpr("CAST(value AS STRING) AS order_json")
      .writeStream.format("console").outputMode("append")
      .option("checkpointLocation", "/tmp/kafka-lab-checkpoint")
      .start())
```

Send more orders using producer.py. Spark console output appears in Jupyter server logs, not necessarily the notebook output. To stop: query.stop(). If “Failed to find data source: kafka” appears, the Spark Kafka connector is missing.

Quick reference — which address?

Kafka CLI INSIDE Kafka container: localhost:19092 | Python on host: localhost:9092 | Jupyter/Spark in Docker: kafka:19092 | Kafka UI: http://localhost:8085

Final practice checklist

☐ One producer and one consumer

☐ One producer and two consumers in different groups

☐ Two consumers in the same group with one partition

☐ Three partitions and independent offsets

☐ Key-based routing

☐ Two consumers in one group across three partitions

☐ Consumer rebalancing

☐ Multiple independent consumer groups

☐ Consumer lag and committed offsets

☐ Kafka UI producer and inspection

☐ Python producer with explicit partitions

☐ Python consumer and Python consumer group

☐ Optional retention and PySpark streaming

Lab note: The commands are aligned with the provided docker-compose.yml. They have not been executed against your local Docker runtime. If a topic already exists with a different partition count, create a new topic name rather than assuming --if-not-exists changes it.